

Restaurant Table Reservation & Billing System

Project Report

Course: Java Programming

Student Name	[DHRUV KUMAR SHARMA]
Registration Number	25BAI10952
Course / Subject	Java Programming
Submission Date	[15-09-2026]

1. Introduction

This report documents the design, implementation, and evaluation of a console-based Java application built to manage the day-to-day front-of-house operations of a small restaurant: table reservations, food ordering, and billing. The project was developed as part of the VITyarthi “Build Your Own Project” flipped-course evaluation for the Java Programming course, applying core object-oriented programming concepts learned throughout the course to a practical, real-world scenario.

2. Problem Statement

Small and mid-sized restaurants often manage table reservations, food orders, and billing manually using paper registers or informal methods. This leads to double bookings, order mix-ups, and billing errors. There is a need for a simple, reliable system to digitize these core front-of-house operations without the overhead of a full commercial point-of-sale system.

3. Functional Requirements

The system is organized into three major functional modules:

3.1 Table Reservation Module

- View all tables and their current availability status
- Book a table by capturing customer name, phone number, and reservation time
- Prevent double-booking — a table already marked BOOKED cannot be re-booked

- Cancel an active reservation, which automatically frees the associated table
- View a complete list of all reservations with their status

3.2 Order Management Module

- Browse a preloaded restaurant menu (item name and price)
- Add one or more menu items with quantities to an order tied to an active reservation
- Finalize an order once all items have been added
- View all orders placed, including line items and subtotal

3.3 Billing Module

- Generate an itemized bill from a finalized order
- Automatically calculate 5% tax on the subtotal
- Optionally apply a 10% loyalty discount
- Mark a bill as paid, which frees the associated table for future reservations
- Persist every generated bill to a text-based transaction log file

4. Non-Functional Requirements

Requirement	How It Is Addressed
Performance	In-memory Java collections (ArrayList) are used for all lookups, giving near-instant response for table/menu/order queries within a single session.
Reliability	Table availability checks prevent double-booking; bills cannot be generated for empty orders.
Usability	A simple numbered console menu guides the user through every operation with clear prompts.
Maintainability	The codebase follows a layered architecture (model / service / data / util), making each concern independently modifiable.
Error Handling	Input validation (Validator class) rejects invalid names, phone numbers, quantities, and menu/table IDs with clear error messages instead of crashing.
Resource Efficiency	Lightweight file-based logging is used instead of a full database, keeping the application dependency-free and fast to run.

5. System Architecture

The application follows a simple layered architecture. The Presentation Layer (Main.java) handles all console input/output. The Service Layer contains the business logic, split across three services matching the three functional modules. The Model Layer defines the core domain entities, and the Data Layer handles persistence of transaction records to a text file. A cross-cutting Validator utility is used by the presentation layer to sanitize user input before it reaches the services.

System Architecture Diagram

Restaurant Table Reservation & Billing System

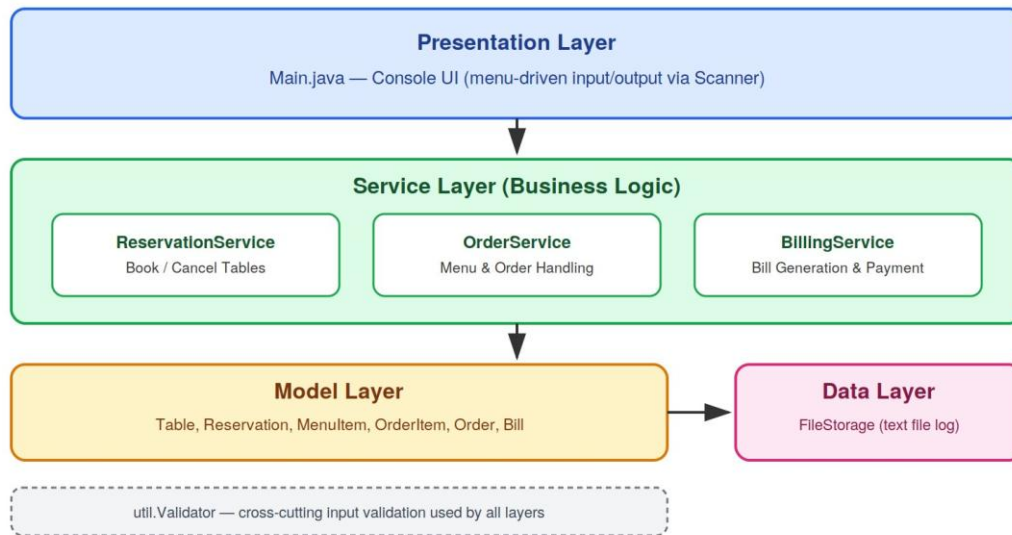


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

The primary actor is restaurant Staff/Waiter, who interacts with the system to manage reservations, orders, and bills. “Generate Bill” includes viewing the order, and “Book a Table” can optionally extend to applying a loyalty discount at billing time.

Use Case Diagram

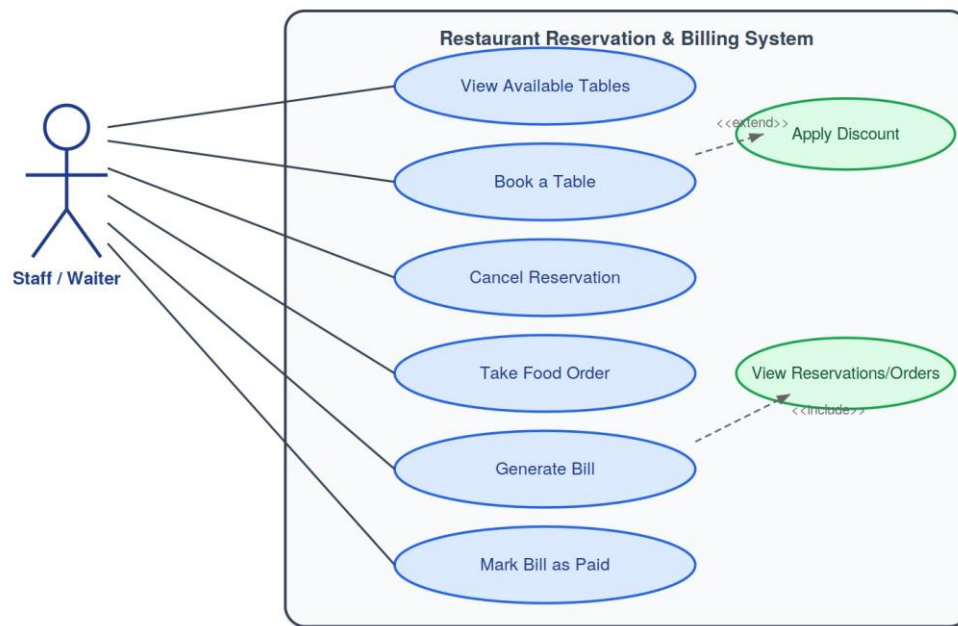


Figure 2: Use Case Diagram

6.2 Class Diagram

The core domain model consists of six classes. A Reservation is linked to exactly one Table. An Order is linked to one Reservation and contains many OrderItem instances, each referencing a MenuItem. A Bill is generated from exactly one Order.

Class Diagram

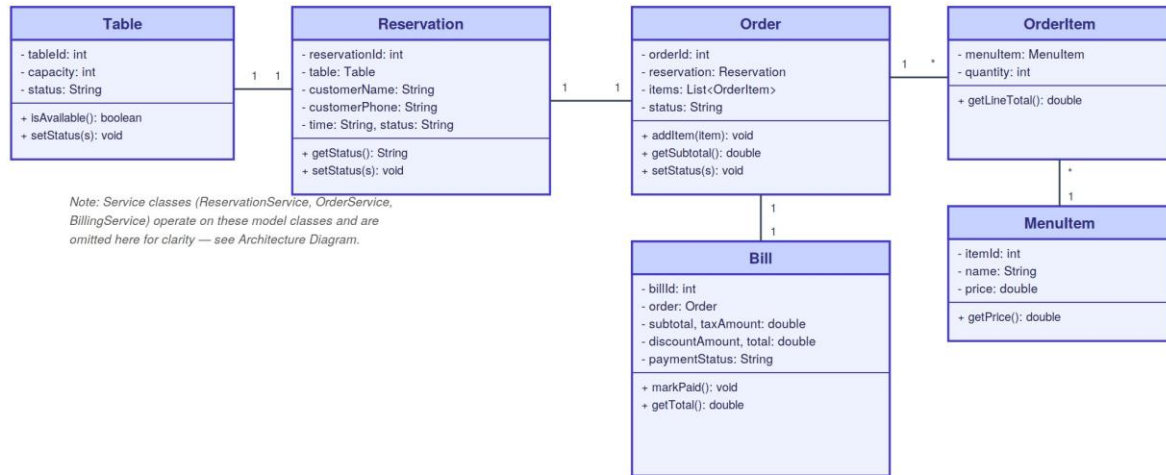


Figure 3: Class Diagram

6.3 Sequence Diagram

The sequence diagram below traces a complete end-to-end flow: booking a table, placing a food order against that reservation, generating a bill, and finally marking the bill as paid (which frees the table again).

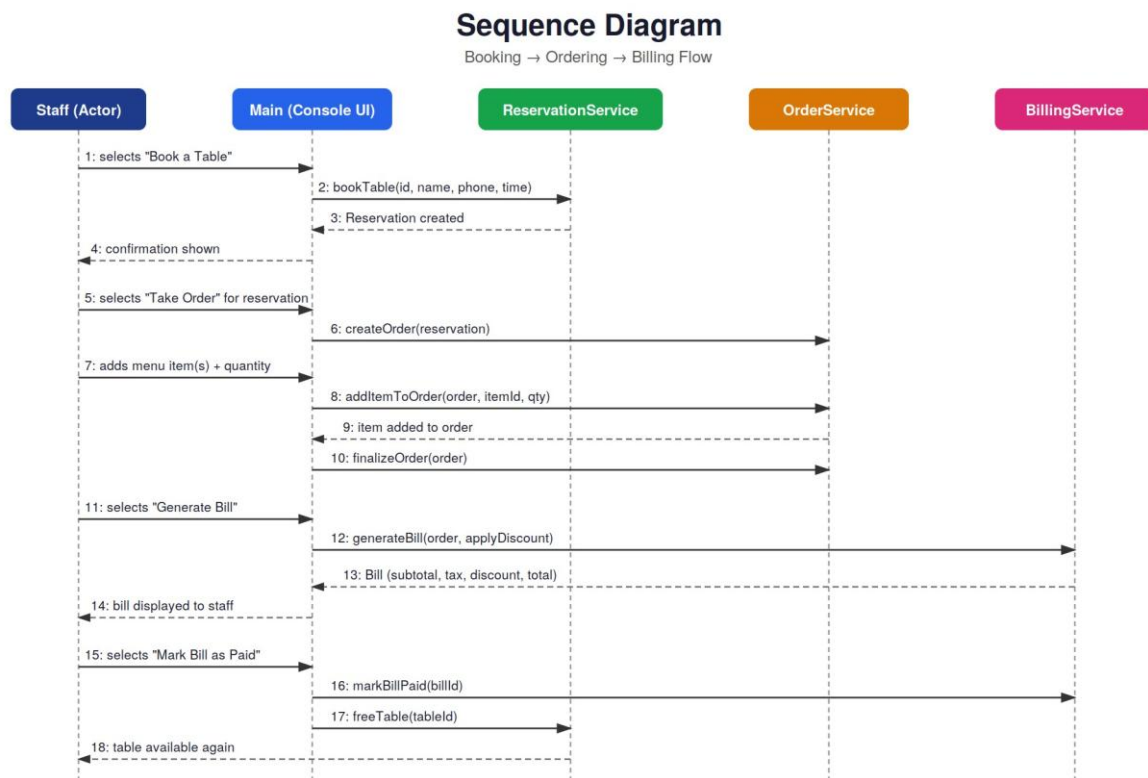


Figure 4: Sequence Diagram

6.4 Process Flow / Workflow

Table.AVAILABLE → Reserve (ReservationService) → Table.BOOKED → Take Order (OrderService) → Finalize Order → Generate Bill (BillingService) → Apply Tax/Discount → Mark Paid → Table.AVAILABLE again.

Note: Since this project uses simple file-based logging rather than a relational database, a formal ER diagram/schema was not required; the transaction log format is documented in Section 9.

7. Design Decisions & Rationale

- **Layered architecture (model/service/data/util):** chosen to keep business logic separate from console I/O, making the code easier to test, extend, or later port to a GUI or web front-end.
- **File-based persistence over a database:** the scope of the project (a single-session console app) did not justify the overhead of setting up and configuring a database; a simple append-only log file satisfies the “proof of transaction” requirement while keeping the project dependency-free.
- **In-memory Collections (ArrayList) as the primary store:** keeps lookups fast and the code simple, appropriate for the expected data volume of a small restaurant during a single run.
- **Centralized input validation (Validator class):** avoids duplicating validation logic across multiple menu flows in Main.java and makes rules easy to change in one place.
- **Status-based state machine for Table/Reservation/Order/Bill:** using simple String status fields (AVAILABLE/BOOKED, ACTIVE/CANCELLED, OPEN/FINALIZED, PAID/UNPAID) keeps the state transitions easy to reason about and display to the user.

8. Implementation Details

The system is implemented in pure Java (JDK 17+, tested on JDK 21) with no external libraries. It consists of 10 source files organized into four packages:

Package	Classes	Responsibility
model	Table, Reservation, MenuItem, OrderItem, Order, Bill	Plain data-holding domain entities with getters/setters and simple derived calculations (e.g., line totals, subtotal).
service	ReservationService, OrderService, BillingService	Business logic: booking rules, order building, tax/discount/bill computation.
data	FileStorage	Appends generated bill summaries to data/transactions_log.txt.
util	Validator	Static helper methods for validating names, phone numbers, quantities, and time input.
(default)	Main	Console menu loop; wires together all three services and handles all user I/O.

Key implementation notes: tax is fixed at 5% and the loyalty discount at 10%, both defined as constants in the Bill class; sample data (6 tables of varying capacity and an 8-item menu) is preloaded at startup in ReservationService and OrderService respectively, and can be edited directly in those files.

9. Screenshots / Results

```
----- BILL #1 -----  
Customer: Ravi Kumar  
Table #: 1  
Subtotal: Rs. 690.00  
Tax (5%): Rs. 34.50  
Discount: Rs. 69.00  
TOTAL: Rs. 655.50  
Payment Status: PAID  
-----
```

10. Testing Approach

The application was manually tested end-to-end by simulating a full customer lifecycle through the console: booking a table, verifying that the same table could not be booked twice, placing a multi-item order, generating a bill with and without the loyalty discount, marking the bill as paid, and confirming the table became available again. Edge cases tested included invalid phone numbers, zero/negative quantities, attempting to bill an empty order, and cancelling a non-existent reservation — all were correctly rejected with descriptive error messages rather than causing the program to crash.

11. Challenges Faced

- Designing a clean way to link Reservations, Orders, and Bills together without a database required careful use of object references rather than foreign keys.
- Deciding where validation should live (UI vs. service layer) to avoid duplicating checks while keeping error messages clear to the console user.
- Handling Scanner input edge cases (e.g., mixing `nextInt()` and `nextLine()` calls) to prevent the console menu from skipping prompts.

12. Learnings & Key Takeaways

- Practical experience structuring a Java project using a layered (model/service/data/util) architecture rather than a single monolithic class.
- Reinforced core OOP concepts: encapsulation (private fields with getters/setters), composition (Order containing OrderItems), and single-responsibility per class.
- Learned to design simple state machines (AVAILABLE/BOOKED, OPEN/FINALIZED, PAID/UNPAID) to model real-world business processes.
- Gained experience with basic file I/O for persistence without the overhead of a full database.

13. Future Enhancements

- Replace file-based logging with a proper relational database (e.g., MySQL via JDBC) for durable, queryable storage across sessions.
- Add a graphical user interface using JavaFX or Swing for a more realistic front-of-house experience. ● Add concurrency-safe table booking (synchronization) to support multiple simultaneous staff terminals.
- Introduce unit tests (JUnit) for the service layer to formalize the manual testing currently performed.
- Add user authentication so different staff members can be tracked per transaction.

14. References

- Oracle Java SE Documentation – docs.oracle.com/en/java